

SQL 注入漏洞报告 (Lab1: WHERE 子句注入)

1. 漏洞名称

SQL Injection (SQL 注入) — WHERE 子句绕过, 检索隐藏数据

2. 漏洞等级

高危 (High)

3. 漏洞描述

该网站商品分类查询功能存在 SQL 注入漏洞。攻击者可以通过修改请求参数, 将恶意 SQL 语句插入后端 SQL 查询语句中, 改变原有查询逻辑, 从而获取数据库中的隐藏数据。

4. 漏洞位置

请求路径: /filter

漏洞参数: category

5. 漏洞复现过程

第一步: 正常请求

访问商品分类 category=Gifts, 页面仅显示 Gifts 分类商品。

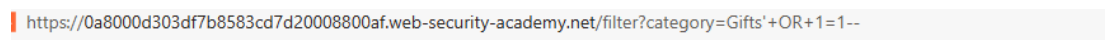
第二步: 抓包与测试 (证据截图)

拦截请求, 将参数修改为 category=Gifts' OR 1=1--。

 `https://0a8000d303df7b8583cd7d20008800af.web-security-academy.net/filter?category=Gifts%27+OR+1=1--`

截图 1: 地址栏中 Payload 的 URL 编码形式

说明: 原始单引号' 被浏览器转为 %27, 空格转为 +, 证明测试字符已带入请求。

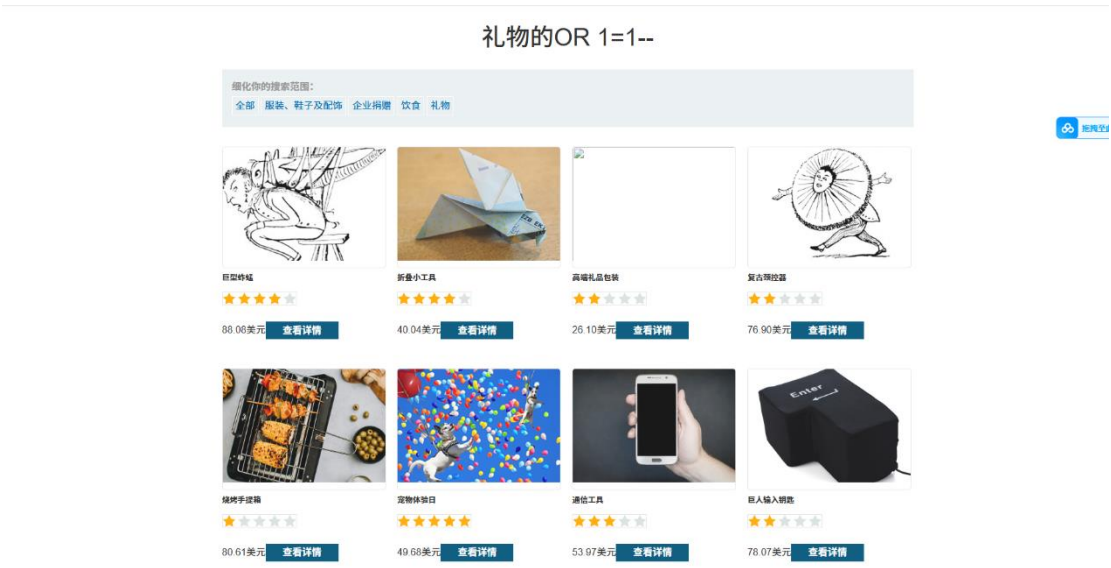
 `https://0a8000d303df7b8583cd7d20008800af.web-security-academy.net/filter?category=Gifts'+OR+1=1--`

截图 2: 浏览器自动解码后的清晰 Payload

说明: Gifts' OR 1=1-- 直接暴露在 URL 中, 说明我们注入的恶意 SQL 语句被原样传

递给了后端。

第三步：漏洞验证（证据截图）



截图 3：漏洞触发结果

说明：页面成功加载出了所有商品（包括原本隐藏的“企业捐赠”和“宠物体验日”等）。证明 $OR\ 1=1$ 让整个 $WHERE$ 条件永远成立，成功绕过了分类限制。



截图 4：实验完成标识

说明：页面顶部显示绿色“实验室 已解决”标识，标志着本 Lab 攻击验证通过。

第四步：漏洞原理

原 SQL：

sql

```
SELECT * FROM products WHERE category='Gifts'
```

经过拼接后执行：

sql

```
SELECT * FROM products WHERE category='Gifts' OR 1=1 -- '
```

- OR 1=1 恒成立，导致查询结果包含所有行。
- -- 是 SQL 注释符，将原本剩余的 ' 与后续逻辑注释掉，语法依旧合法。

6. 漏洞影响

攻击者可能：

- 绕过查询限制，获取全部商品信息（包括未发布/隐藏商品）。
- 进一步利用 UNION 查询读取数据库敏感数据（表名、列名、用户表）。
- 在严重情况下结合其他漏洞导致数据库泄露、权限提升。

7. 修复建议

① 使用参数化查询（推荐）

错误：sql = "SELECT * FROM products WHERE category='" + category + "'"

正确：sql = "SELECT * FROM products WHERE category=?", 并使用参数绑定的方式执行。

② 输入过滤

限制用户输入特殊字符：'、--、#、;。

③ 最小数据库权限

数据库账号不要使用管理员权限（如 root/sa）。

8. 工具

- Burp Suite（抓包、重放）
- Firefox（浏览器调试）
- PortSwigger Web Security Academy（实验平台）